

Windows

A desktop program in SPRFST is an `app` . You describe what the window contains and what each control does; the runtime keeps the two in step.

The whole shape of an app

```
app "Counter" {
  var count = 0
  var step = 1

  window {
    title: "SPRFST Counter"
    width: 420
    height: 280

    title "Count is {count}"
    divider
    button "Add {step}" {
      on click {
        count = count + step
      }
    }
    button "Bigger steps" {
      on click {
        step = step * 2
      }
    }
    button "Reset" {
      on click {
        count = 0
        step = 1
      }
    }
  }
}
```

Three things are going on.

`var count = 0` inside the `app` is **state** . It lives as long as the window does, and any part of the app may change it.

A label with a `{hole}` in it is **bound** . When the state inside the hole changes, that label is redrawn—you never refresh anything by hand.

`on click { ... }` is ordinary SPRFST code. It can call functions, read files, start tasks. There is no separate event language.

The elements

Element	What it is	Properties
window	the frame everything sits in	title, width, height
title / heading	large text	the label
text	ordinary text	the label
button	a button	on click
field / input	one line of typing	value, on change
toggle / check	a tick box	value, on change
list	a list of values	items
divider	a rule between things	—

Reading what the person typed

An event can carry a value. Name it in brackets after the event and it arrives as an ordinary parameter:

```
app "Notes" {
  var draft = ""
  var notes: [Text] = []
  var pinned = false

  window {
    title: "Notes"
    width: 460
    height: 420

    title "{notes.len()} notes"
    field "New note" {
      value: draft
      on change(text) {
        draft = text
      }
    }
    button "Keep it" {
      on click {
        if draft.trim().len() > 0 {
          notes.push(draft.trim())
          draft = ""
        }
      }
    }
  }
  toggle "Pin this window" {
    value: pinned
    on change(ticked) {
      pinned = ticked
    }
  }
  divider
```

```

        list {
            items: notes
        }
    }
}

```

`value: draft` sends state *into* the control, `on change(text)` brings the answer back out. Clearing `draft` in the button handler empties the box, because the binding runs again after every event.

Where the window comes from

The same program runs against more than one backend, chosen with the `SPRFST_UI` environment variable:

Value	What you get
<code>term</code>	the window drawn in the terminal, with numbered controls
<code>json</code>	one line of JSON describing the window, then exit
<code>none</code>	build the window, print its description, exit
<code>unset</code>	<code>term</code> when the terminal is interactive, otherwise <code>none</code>

On a Mac, and inside SPRFST Studio, the app is hosted in a real AppKit window. The terminal backend exists so an interface can be driven on a machine with no screen—which is how every window in this guidebook is tested.

```
SPRFST_UI=term sprfst run examples/18-gui-counter.spf
```

Building an interface by hand

`app` is sugar over `std.ui`, which you can call directly when the interface is generated rather than written out. `ui.window` gives you a node id, `ui.node` adds children, and `ui.on` attaches handlers.

```

use std.io
use std.ui

var clicks = 0

fn main() {
    let w = ui.window("Built by hand", 360, 200)
    let label = ui.node(w, "title", "0 clicks")
    let press = ui.node(w, "button", "Click me")

    ui.on(press, "click", fn() => {
        clicks = clicks + 1
        ui.set(label, "text", "{clicks} clicks")
    })

    ~~ `ui.emit` fires a handler from code, which is how you test an
    ~~ interface without a person pressing anything

```

```
    ui.emit(press, "click")
    ui.emit(press, "click")
    io.say("the label now says: {ui.get(label, "text")}")
    io.say("backend: {ui.backend()}")

    ui.run()
}
```

Note `var clicks` outside `main`. A closure captures the values around it as they were, so a counter that has to survive between clicks lives in app state or in a module level `var`, not in a local.

Exercise

Add a "Remove last" button to the notes app, and a heading that shows the most recent note, or `nothing yet` when the list is empty.